

UE INFORMATIQUE

Programmation 2 : Approfondissement

Laureen Fabre



PROGRAMME

- Pointeurs
 - Allocation automatique
 - Allocation dynamique
- Listes chaînées
- Algorithmes de tri
- Structures de données avancées

LES POINTEURS

COPIES

- On a vu que le système fait une copie des variables quand on les passe à une fonction.
- Comment faire pour que ce ne soit pas le cas ?
- Autre question, comment retourner plusieurs valeurs ?
- Nous allons pouvoir faire cela grâce à des **pointeurs**.

POINTEUR

- Les **pointeurs** sont des variables qui pointent sur une adresse mémoire.
- Tous les pointeurs utilisent la même quantité d'octets : 8 sur une architecture 64 bits.
- Les pointeurs ont un type prédéfinis comme pour les variables vu jusqu'à présent.
- Pour déclarer un pointeur il faut rajouter une *
- Par exemple, pour déclarer un pointeur d'entiers nommé *pentier*, il faut écrire :
`int * pentier ;`

POINTEUR

```
int main()
{
    int ent = 2;
    int* pentier = &ent;

    printf("%d\n", ent);
    printf("%p\n", &ent);
    printf("%p\n", pentier);
    printf("%p\n", &pentier);
    printf("%d\n", *pentier);

    printf("\n%llu\n", sizeof(pentier));
}
```

Console de débogage

```
2
000000FB6F4FFA04
000000FB6F4FFA04
000000FB6F4FFA28
2
8

Sortie de C:\Use
processus 257447
```

192	pentier	200
196		
200	ent	2

POINTEUR

- Attention si l'on déclare le pointeur sans l'initialiser, on pointe n'importe où.
=> *pentier* pourrait pointer sur une adresse mémoire utilisée mais qui n'a absolument rien à voir.
- Une chose primordiale à faire lorsque l'on manipule des pointeurs, c'est les **initialiser** à « rien », afin d'être certain qu'ils ne pointent **nulle part**.
*int * pentier = NULL;*
- Si maintenant l'on souhaite qu'il pointe sur une variable existante, il faut utiliser l'adresse de cette variable grâce au &.
int age = 32;
*int * pentier = &age ;*

POINTEUR

- Si l'on souhaite afficher *pentier*, nous verrons **l'adresse** de la variable *age*.
- Si l'on veut connaître l'adresse du pointeur, c'est comme pour une variable classique il faut utiliser **&**.
&pentier -> retourne l'adresse de pentier
- Si l'on souhaite connaître la **valeur** de la variable sur laquelle on pointe, il faut utiliser *****. On dit qu'on **déréférence** le pointeur.
**pentier -> retourne 32*

LES ALLOCATIONS

ALLOCATION MÉMOIRE

- Au cours de l'exécution d'un programme il faut que de la **mémoire** soit réservée pour fonctionner correctement.
- Il existe deux mécanismes pour allouer de la mémoire : **automatique** et **dynamique**.
- La mémoire est divisée en deux parties : la **pile** et le **tas**.
- Les allocations **automatiques** se font sur la **pile** et les **dynamiques** sur le **tas**.

ALLOCATION AUTOMATIQUE

- C'est ce que nous avons fait jusqu'à présent.
- Lorsque l'on déclare une variable, elle est *allouée* dans la **pile** *automatiquement*.
- De la même manière, elle va être *désallouée* automatiquement lorsque l'on sort du bloc dans lequel elle a été définie. On dit qu'il y a *dépilement*.
- Pour accéder à l'adresse en mémoire d'une variable, il faut mettre **&** devant.

ALLOCATION AUTOMATIQUE

- Si on affiche la pile sur 4 octets et que l'on suppose que l'adresse de la pile commence à 160 alors on a :
- &ent = l'adresse de ent donc dans notre exemple 160
- &s vaut 104
- etc ...

```
int main()
{
    int ent = 2;
    float fl = 4.2f;
    double db = 5.35;
    char c = 'A';
    char s[40] = "ma phrase";

    for (int i = 0; i < 2; i++)
    {
        ent += 2;
    }

    printf("%llu\n", sizeof(int));
    printf("%llu\n", sizeof(float));
    printf("%llu\n", sizeof(double));
    printf("%llu\n", sizeof(char));
    printf("%llu\n", sizeof(s));
}
```

PILE



...					
100	i	2 2			
104	s	m	a		p
108		h	r	a	s
112		e	\0		
...					
144					
148	c	A			
152	db	5.35			
156	fl	4.2			
160	ent	2 6			

4
4
8
1
40
Sort
proc

ALLOCATION DYNAMIQUE

- L'allocation dynamique se fait par le biais de **pointeurs**.
- Lorsque l'on déclare un pointeur, il est *alloué* dans le **tas** *dynamiquement*.
- L'allocation dynamique se fait à l'aide de la fonction **malloc**.
- Comme on alloue nous même la mémoire, c'est également à nous de la *désallouer* grâce à la fonction **free**.
- Il faudra inclure *#include <stdlib.h>*

ALLOCATION DYNAMIQUE D'UN POINTEUR

- Pour le moment nous n'avons pas alloué *dynamiquement* de la mémoire au pointeur. Pour se faire il faut utiliser la fonction **malloc** :
*int *pentier = malloc(sizeof(int)) ;*
- **malloc** va demander à la mémoire de lui allouer de la place pour un entier.
- Mais cette fois-ci, comme on demande une nouvelle zone, il n'y a pas de variable qui y est associée et dont on peut connaître la valeur.
- Pour pouvoir y accéder et la modifier, il faut *déréférencer* le pointeur avec ^{*}
**pentier = 2 ;*

DÉSALLOCATION DYNAMIQUE D'UN POINTEUR

- Comme on se charge de la création de ce pointeur, c'est également à nous de nous charger de sa **destruction** lorsque l'on sait que l'on n'en a plus besoin.
- Là aussi c'est primordial de le faire car sinon on peut se retrouver avec la mémoire surchargée et le programme plantera.
- Après avoir *free* le pointeur, on rend l'adresse mémoire à l'ordinateur. Vous n'avez plus le droit de l'utiliser, **mais** rien ne vous en empêche, la variable est toujours *accessible*, or l'adresse a pu être réattribuée à une autre variable totalement différente.
- Il faut donc tout de suite après l'avoir *free*, le faire pointer *nulle part*.

```
free( pentier );  
pentier = NULL;
```

ALLOCATION DYNAMIQUE D'UN POINTEUR

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int* pentier = malloc(sizeof(int));
    printf("\n\n\npentier %p\n", pentier);
    printf("&pentier %p\n", &pentier);
    printf("*pentier%d\n", *pentier);

    *pentier = 6;

    printf("\n\n\npentier %p\n", pentier);
    printf("&pentier %p\n", &pentier);
    printf("*pentier%d\n", *pentier);

    free(pentier);

    printf("\n\n\npentier %p\n", pentier);
    printf("&pentier %p\n", &pentier);
    printf("*pentier%d\n", *pentier);

    pentier = NULL;

    printf("\n\n\npentier %p\n", pentier);
    printf("&pentier %p\n", &pentier);
    printf("*pentier%d\n", *pentier);
}
```

Console de débogage Micros...

```
pentier 000001DABA626B30
&pentier 0000002B1D14F598
*pentier-842150451

pentier 000001DABA626B30
&pentier 0000002B1D14F598
*pentier6

pentier 000001DABA626B30
&pentier 0000002B1D14F598
*pentier-572662307

pentier 0000000000000000
&pentier 0000002B1D14F598

Sortie de C:\Users\laure\
processus 13920). Code :
Appuyez sur une touche po
```

TAS

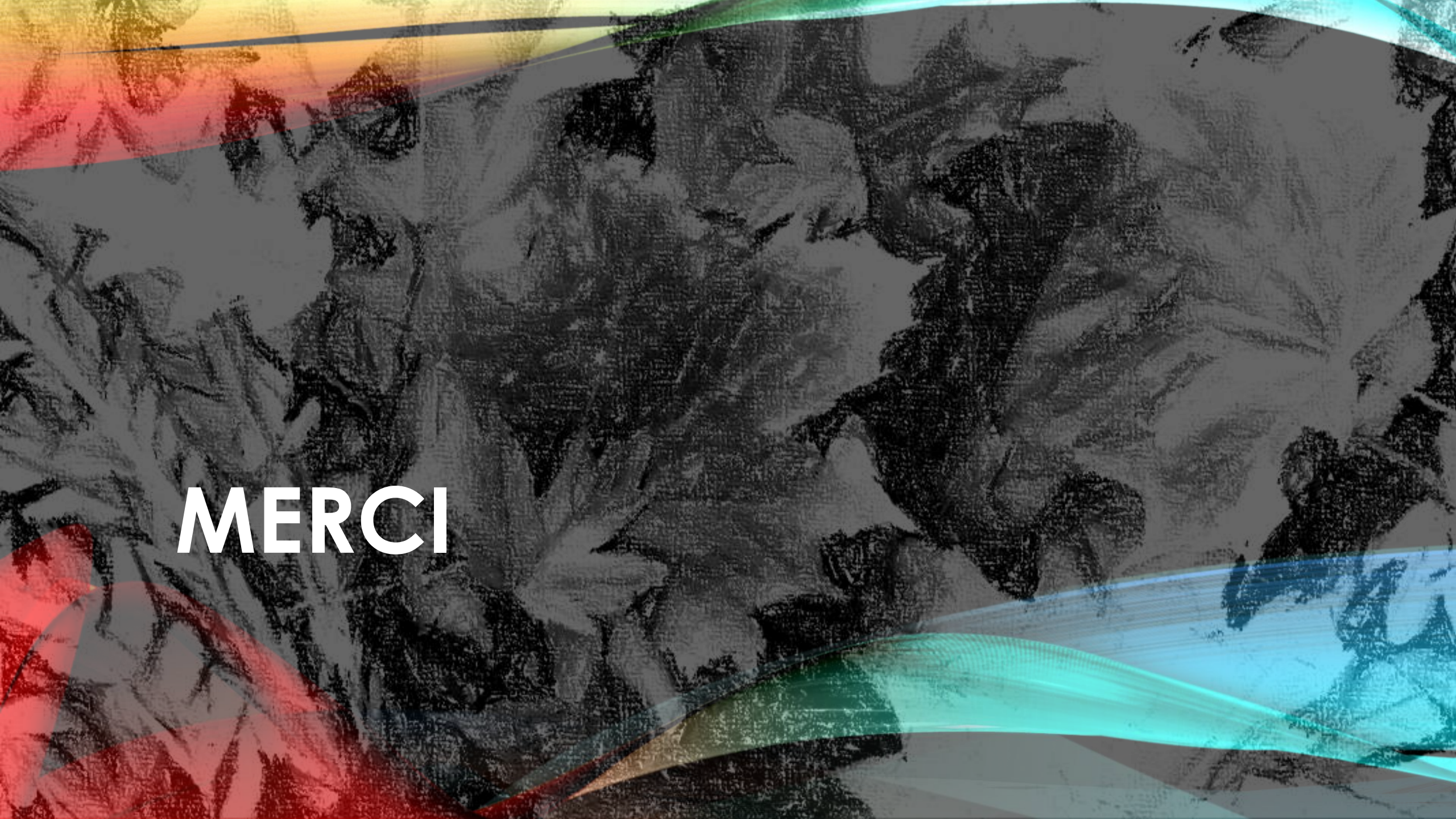


488	int	? 6
...		
508	pentier	488
512		
...		

MODIFIER L'ALLOCATION DYNAMIQUE D'UN POINTEUR

- Si l'on veut changer la taille allouée pour un pointeur, il faut utiliser la fonction **realloc** :

```
int * pentier = malloc(sizeof(int)) ;  
pentier = realloc(pentier, 5*sizeof(int));
```

MERCI